Assignment 3: Evaluation Plan

Overview

This document outlines the comprehensive evaluation plan for assessing the **Dynamic Emergency Response Allocation Agent** on the *RescueBench* environment. It details our research objectives, quantitative metrics rooted in Operations Research, baseline comparisons, experimental procedures, and strategies for benchmark validation. This protocol ensures a rigorous, reproducible assessment of the agent's spatial reasoning, constraint satisfaction, and dynamic adaptability in simulated crisis scenarios

Part 1: Evaluation Objectives - Trusha

Research Questions

Our evaluation seeks to answer the following research questions:

RQ1: Does the proposed agent architecture outperform baseline approaches on complex resource allocation tasks?

We compare the Dynamic Emergency Response Allocation Agent against two baselines:

- Greedy heuristic solver (nearest-resource assignment)
- Single-shot LLM planner without validation or repair loops

This comparison evaluates whether the agent's **planning, tool interaction, and validation loop improve solution feasibility and overall performance**.

RQ2: How does agent performance change as task complexity increases?

RescueBench contains four tiers of difficulty:

- Tier 1: Basic triage and routing
- Tier 2: Constraint satisfaction
- Tier 3: Ethical prioritization under resource scarcity
- Tier 4: Dynamic replanning with mid-mission updates

We evaluate how performance varies across these tiers to determine whether the agent can handle **increasingly complex planning requirements**.

RQ3: How effectively can the agent adapt to dynamic changes in the environment?

Dynamic scenarios introduce unexpected updates such as:

- road closures
- vehicle failures
- hospital capacity changes

These updates require the agent to **re-evaluate its plan and reallocate resources**. We measure how often the agent successfully adapts to these changes without violating constraints.

RQ4: What types of tasks does the agent solve well versus poorly?

We analyze failure cases to understand:

- which constraints most frequently cause errors
- whether certain incident types are harder to resolve
- whether dynamic updates significantly reduce success rates

This analysis helps identify **limitations of LLM-based planning systems in operations research environments**.

Success Criteria

The evaluation will consider the approach successful if the following outcomes are observed:

High overall task success rate

The agent should successfully complete a large proportion of the 20 benchmark scenarios while satisfying all hard constraints.

Improved performance relative to baselines

The agent should outperform baseline methods on key metrics such as:

- incident resolution score
- constraint violation rate
- dynamic replanning success

Low hallucination or constraint violation rate

Since the architecture includes a deterministic validator, the agent should demonstrate **near-zero physically impossible actions**.

Stable performance across task tiers

While performance may decline as difficulty increases, the agent should still perform meaningfully on complex Tier 3 and Tier 4 scenarios.

Negative or Inconclusive Results

The evaluation would be considered negative or inconclusive if:

- The agent performs similarly to or worse than the greedy heuristic baseline.
- The validation loop fails to significantly reduce constraint violations.
- Performance degrades dramatically on dynamic tasks.
- The agent frequently produces infeasible or invalid action plans.

Such outcomes would suggest that the LLM planning component does not meaningfully improve resource allocation decisions compared to simpler deterministic approaches.

Part 2: Benchmark Evaluation Metrics - Trusha

Primary Metrics

- **Task Success Rate:** Percentage of tasks completed successfully
- **Task Completion Quality:** Quality ratings or scores for completed tasks
- **Efficiency Metrics:** Time, API calls, tokens used, cost per task
- **Error Metrics:** Types and frequencies of errors

1. Priority-Weighted Resolution Score (Primary Success Metric)

This metric measures how effectively the agent resolves incidents while prioritizing the most severe emergencies.

Each incident is assigned a severity weight between 1 and 10. The score is computed as:

$$S = \sum_{i=1}^{n} (w_i \times I_i)$$

Where:

- w_i is the severity weight of incident *i*
- I_i is a binary variable indicating whether the incident was successfully resolved before its deadline

Interpretation

- Higher scores indicate that the agent successfully resolved more high-priority incidents.
- A perfect score corresponds to resolving all incidents before their deadlines.

Aggregation

Scores are averaged across all 20 benchmark scenarios.

2. Response Efficiency

This metric evaluates how close the agent's routing decisions are to the optimal shortest-path solution.

Where:

$$E = \frac{T_{optimal}}{T_{agent}}$$

- $T_{optimal}$ is the minimum travel time computed using Dijkstra's algorithm
- T_{agent} is the travel time produced by the agent's dispatch plan

Interpretation

- E=1.0E = 1.0E=1.0 indicates optimal routing.
- E<1.0E < 1.0E<1.0 indicates suboptimal routing.

Aggregation

Efficiency scores are averaged across incidents and scenarios.

3. Hallucination / Constraint Violation Rate

This metric counts the number of invalid actions proposed by the agent.

Examples include:

- dispatching vehicles without required capabilities
- exceeding vehicle capacity limits
- routing through blocked roads
- ignoring hospital capacity constraints

Calculation

$$V = \sum c_{invalid}$$

Where $c_{invalid}$ counts each invalid action detected by the validator.

Interpretation

- A score of **0 indicates perfect constraint adherence**
- Higher scores indicate more hallucinated or infeasible actions

4. Task Success Rate

Task success rate measures the percentage of benchmark scenarios successfully completed.

A task is considered successful if:

- all required constraints are satisfied
- the final resolution score exceeds a minimum threshold
- no unrecoverable constraint violations occur

Calculation

$$TaskSuccess = \frac{\text{Number of successful tasks}}{20}$$

Secondary Metrics (if applicable)

- **Robustness:** Performance on edge cases or adversarial inputs
- **Consistency:** Variance in performance across multiple runs
- **Generalization:** Performance on held-out or modified tasks
- **Human Evaluation:** User satisfaction, preference ratings

1. Robustness to Dynamic Updates

This metric measures performance on scenarios containing mid-mission updates.

Examples include:

- bridge collapse
- new incident appearance
- vehicle breakdown

The metric measures the percentage of dynamic tasks that the agent successfully replans for.

2. Consistency Across Runs

Since LLM outputs can vary between runs, we will evaluate the agent multiple times per scenario.

Measurement

- run each scenario 3–5 times
- compute variance in success rate and resolution score

Low variance indicates stable planning behavior.

3. Efficiency Metrics

We also track computational cost:

- number of LLM calls per task
- tokens consumed
- estimated API cost per task

These metrics measure **practical deployability of the agent architecture**.

Metric Aggregation

Final benchmark results will be reported using the following aggregated statistics:

- Average Priority-Weighted Resolution Score
- Average Response Efficiency
- Total Constraint Violations
- Task Success Rate
- Dynamic Task Success Rate

These metrics provide a comprehensive view of agent performance across **correctness, efficiency, and robustness**.

Part 3: Baseline Comparisons - Akhil

Baseline Approaches

1. **Standard Zero-Shot LLM**: The same Gemini model used in our agent, prompted once with the full scenario and asked to produce a complete dispatch plan in a single pass. No tools, no validator feedback, no self-correction loop.
    a. **Relevance**: Isolates the contribution of our ReAct architecture — if this baseline scores comparably, our tooling adds little value.
    b. **Implementation**: Remove tool definitions and orchestration loop from our existing codebase.
2. **Ablated Agent**: Our full architecture with dynamic replanning stripped out. Processes the initial scenario state but ignores mid-mission dynamic events (like flooding, hospital capacity changes). Still uses tools and the validator.
    a. **Relevance**: Directly tests whether real-time adaptability provides measurable benefit over a static planner, which is the core claim of our design.
    b. **Implementation**: Feature flag in the existing Agent class that disables dynamic event processing.

Part 4: Experimental Setup

Test Procedure

- **How will you run each agent on each task?**
  Each agent is given a scenario JSON file, automatically solves it by calling tools and dispatching vehicles, and outputs a score

- **How many trials per task-agent combination?**
  LLM-backed agents would run 3-5 times per scenario and then the scores are averaged. But rule-based agents would only run once since they always produce the same answer.

- **Will you randomize order to control for effects?**
  No, every agent sees scenarios in the same fixed order (easy to hard) so results are directly comparable.

- **How will you handle non-deterministic behavior?**
  The model's temperature is set to 0 for consistency, and averaging the scores from 3 rounds addresses any remaining variability.

Data Collection

- **What data will you log for each trial?**
  We'll record: scenario, agent, attempt number, final PWRS score, violation count, steps taken, and tokens used.

- **How will you store and organize results?**
  Each trial saves its own JSON file, which an aggregation script combines into a summary spreadsheet (if results grow large, we may migrate to a SQL database for easier filtering and querying).

- **What tools or infrastructure will you use?**
  Everything runs locally in Python against the Gemini API, with all agents sharing the same *simulation* code to ensure same testing conditions.

Part 5: Benchmark Validation

Task Quality

- **How will you ensure tasks are clear and well-specified?**
  All tasks use a strictly standardized JSON schema. We utilize a custom Python script (*visualize_city.py*) to render the JSON into a NetworkX graph image, visually verifying that all nodes connect and no impossible map geographies exist. Furthermore, our *WorldState* Python parser enforces the schema and throws explicit errors for malformed task files.

- **How will you verify that tasks are appropriately challenging?**
  We will use our baselines as difficulty calibration tools. If our deterministic Dijkstra baseline cannot find a valid mathematical path, the scenario is flagged as impossible. Conversely, if our "Standard Zero-Shot LLM" baseline passes a complex Tier 4 scenario perfectly on the first try, the task is too easy. We will iteratively tighten time windows or add stricter weight limits until advanced agentic reasoning is genuinely required.

- **Will you pilot test with human subjects?**
  We will not use external human subjects. Instead, team members will conduct internal pilot testing by manually attempting to solve a subset of scenarios to ensure valid, logical solutions exist and that time windows (*deadline_min*) are physically achievable.

Benchmark Coverage

- **Do your tasks cover different aspects of the domain?**

Yes. The 20 JSON scenarios cover structural fires, transit accidents, natural disasters (flooding, earthquakes), and civil infrastructure crises. Each requires specific combinations of our heterogeneous fleet (ambulances, fire engines, police cars, supply trucks), forcing the agent to handle diverse medical, physical, and logistical challenges.

- **Is there appropriate difficulty variation?**
  Yes, difficulty scales across our 4-tier system (5 tasks each):
    1. Tier 1: Basic triage and routing.
    2. Tier 2: Constraint satisfaction (weight limits, capacity matching).
    3. Tier 3: Ethical prioritization (resource scarcity).
    4. Tier 4: Dynamic replanning (mid-mission graph updates).

- **Are there enough tasks to draw meaningful conclusions?**
  Yes. Because we run the LLM 3-5 times per scenario (as outlined in our experimental setup) to account for non-determinism, our 20 tasks will generate 60-100 fully logged, multi-step execution traces per agent. This provides a statistically meaningful sample size to evaluate logic and constraint adherence without prohibitive API costs.

Benchmark Reliability

- **How will you verify that evaluation is consistent?**
  All agents (our proposed agent and baselines) interface with the exact same Python simulation environment (*WorldState*) and are checked by the identical deterministic safety layer (*ValidatorTool*). There is no subjective human grading; every action is programmatically evaluated.

- **Can results be reproduced?**
  Yes. As established in our test procedure, we set the LLM temperature to 0 and run 3-5 trials per task, averaging the results to smooth out variability. Because our benchmark consists of 20 static JSON files, anyone with our Python scripts and a Gemini API key can recreate the exact environmental states and perfectly reproduce our findings.

- **Are success criteria objective and unambiguous?**
  Yes, they are rooted in strict Operations Research math. The Priority-Weighted Resolution Score (PWRS) is an algebraic formula, Response Efficiency compares travel time directly against Dijkstra's mathematical absolute, and Constraint Violations are objective boolean triggers evaluated by the Python validator.

Part 6: Limitations and Threats

Known Limitations

- **What aspects of agent performance won't your evaluation capture?**
  Our benchmarks include a world created by ours, it is difficult to know how much this would relate to real world mechanics before actually deploying.

- **What real-world considerations are missing from your benchmark?**
  Things like timing, administrative details, dynamic world changes (traffic, problems with vehicles, unpredictable world events) that make it difficult to know how well this could generalize to the real world.

- **What biases might exist in your task design?**
  Creating a world by ourselves without consulting with experts that know more about the topic would result in biases because we might not take into consideration different problems, and known best practices when developing solutions.

Threats to Validity

- **What factors might confound your results?**
  Different city setups, and different ways to organize a city / world could make one model be better than the other.

- **When might your results not generalize?**
  Expanding from before they might not generalize accurately on real world implementation. It is difficult to know without first deploying because there could always be unexpected events we are not taking into account

- **What assumptions are you making?**
  We are assuming that the variables chosen to describe the world are sufficient and that there are no unpredictable world events: such as people getting sick, unexpected unavailability of resources, etc.

Mitigation Strategies

- **How will you address or control for these limitations?**
  We have to deploy on the real world, and then see how much our created world correlates with the real world. Then we might have to optimize and change for better accuracy and needs.

- **What caveats will you include in your conclusions?**

That this benchmark only works with the world we created, and might not necessarily mean that one model is better than another in the real world. The benchmark is only used as an approximation.